

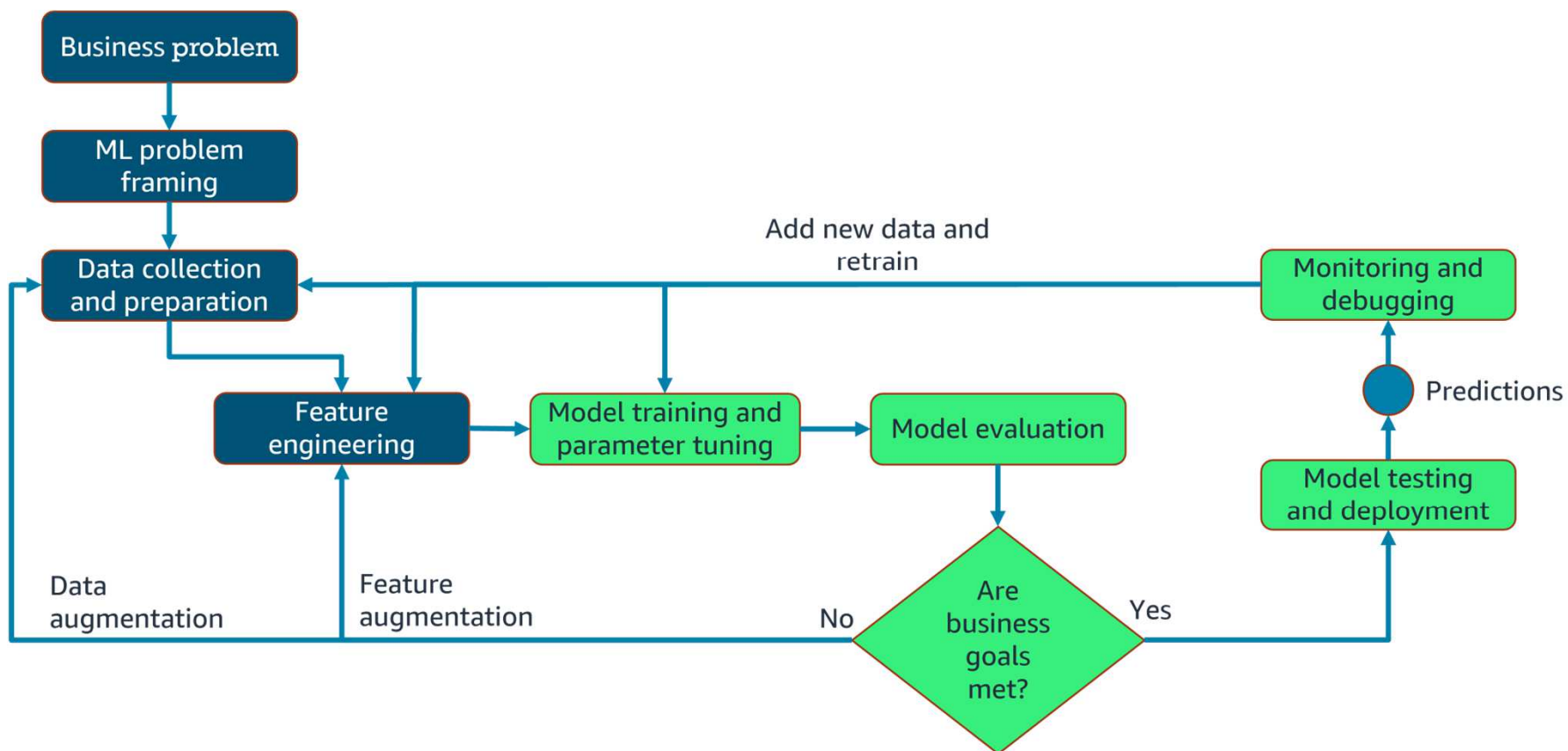


**Developing
machine
learning
solutions**

Introduction

- In this machine learning course, you will learn about the machine learning lifecycle, and how to use AWS services at every stage. Additionally, you will discover the diverse sources for machine learning models and learn techniques to evaluate their performance. You will also understand the importance of machine learning operations (MLOps) in streamlining the development and deployment of your machine learning projects.

ML Development Lifecycle



Developing ML Solutions with Amazon SageMaker

Amazon SageMaker is a fully managed ML service. In a single unified visual interface, you can perform the following tasks:

- Collect and prepare data.
- Build and train machine learning models.
- Deploy the models and monitor the performance of their predictions.

AWS SageMaker

Collecting, analyzing, and preparing your data

- **Amazon SageMaker Data Wrangler**

Low-code no-code (LCNC) tool. It provides an end-to-end solution to import, prepare, transform, featurize, and analyze data by using a web interface.

- **Amazon SageMaker Studio Classic**

Built-in integration of **Amazon EMR** and **AWS Glue interactive sessions** to handle large-scale interactive data preparation and machine learning workflows

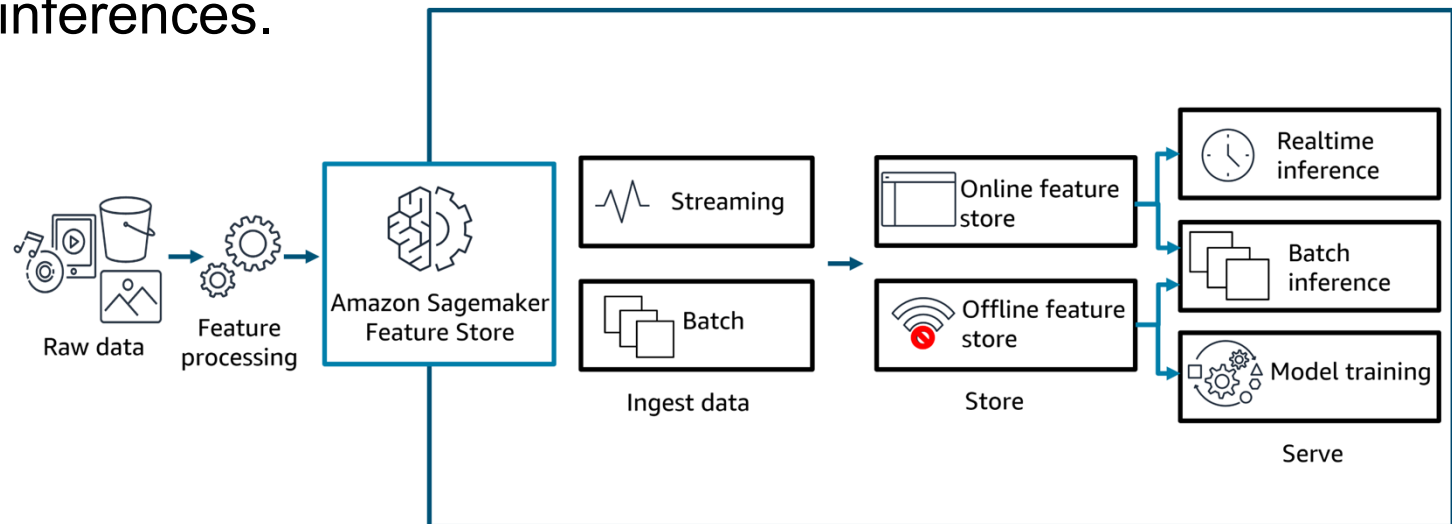
- **SageMaker Processing API**

Run scripts and notebooks to process, transform, and analyze datasets . Supports various ML frameworks such as scikit-learn, MXNet, or PyTorch while benefiting from fully managed machine learning environments

AWS SageMaker Managing Features

Amazon SageMaker Feature Store helps data scientists, machine learning engineers, and general practitioners to create, share, and manage features for ML development.

Features stored in the store can be retrieved and enriched before being served to the ML models for inferences.



AWS SageMaker Model Training and Evaluation

SageMaker provides a training job feature to train and deploy models using built-in algorithms or custom algorithms.

SageMaker launches the ML compute instances and uses the training code and the training dataset to train the model. It saves the resulting model artifacts in an Amazon Simple Storage Service (Amazon S3) bucket that can be used later for inference.

Customers aiming at a LCNC option can use **Amazon SageMaker Canvas**. With SageMaker Canvas, they can use machine learning to generate predictions without needing to write any code.

Amazon SageMaker JumpStart provides pretrained, open source models that customers can use for a wide range of problem types.

AWS SageMaker Model Evaluation

Amazon SageMaker Experiments to experiment with multiple combinations of data, algorithms, and parameters, all while observing the impact of incremental changes on model accuracy.

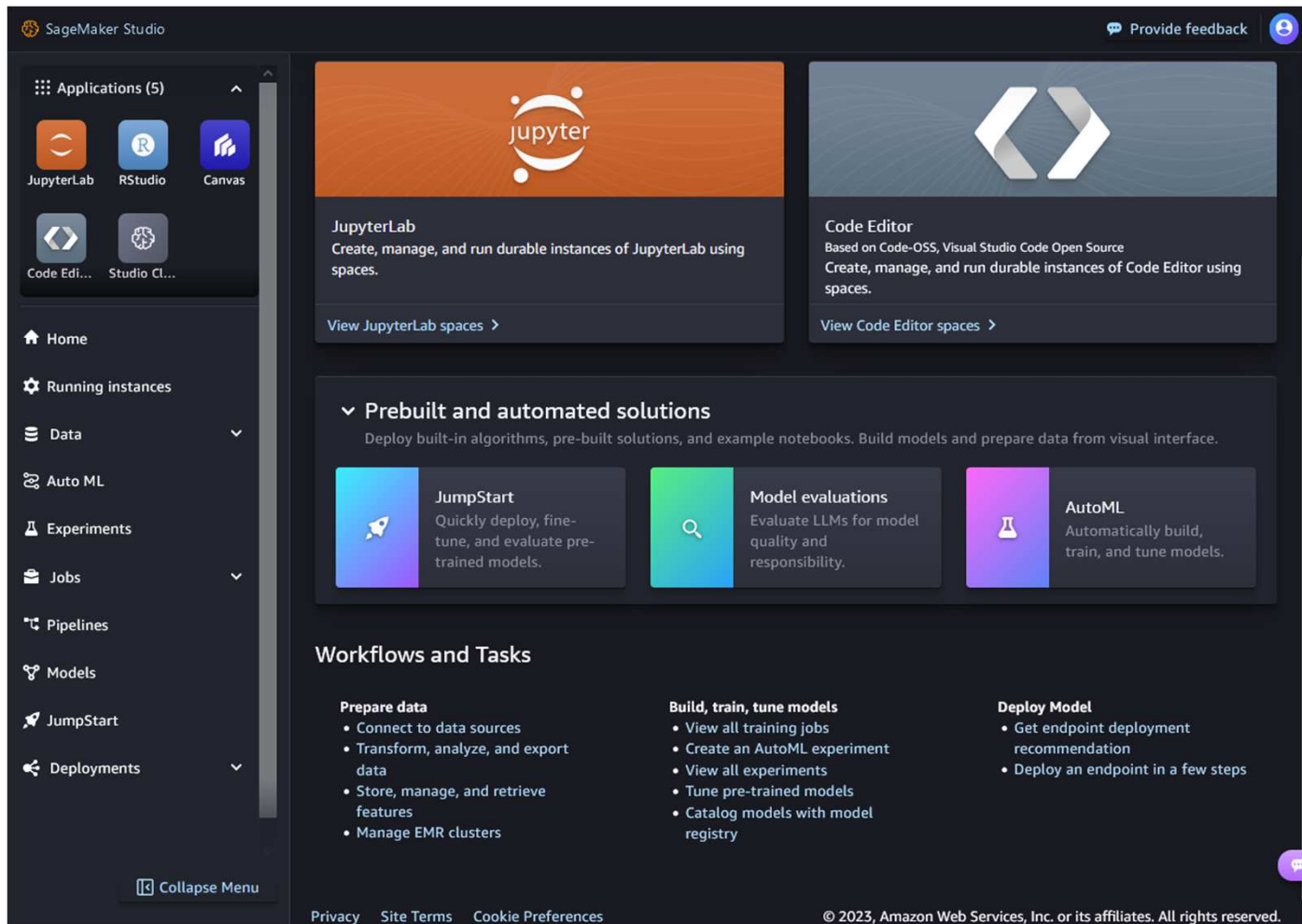
Hyperparameter tuning is a way to find the best version of your models. **Amazon SageMaker Automatic Model Tuning** does that by running many jobs with different hyperparameters in combination and measuring each of them by a metric that you choose.

AWS SageMaker Deployment and Monitoring

Customers can deploy their ML models to make predictions, also known as **inference**. **SageMaker** provides a broad selection of ML infrastructure and model deployment options to help meet all your ML inference needs.

Amazon SageMaker Model Monitor, customers can observe the quality of SageMaker ML models in production. They can set up **continuous monitoring or on-schedule monitoring**. SageMaker Model Monitor helps **maintain model quality** by detecting violations of user-defined thresholds for **data quality, model quality, bias drift, and feature attribution drift**.

AWS SageMaker Studio

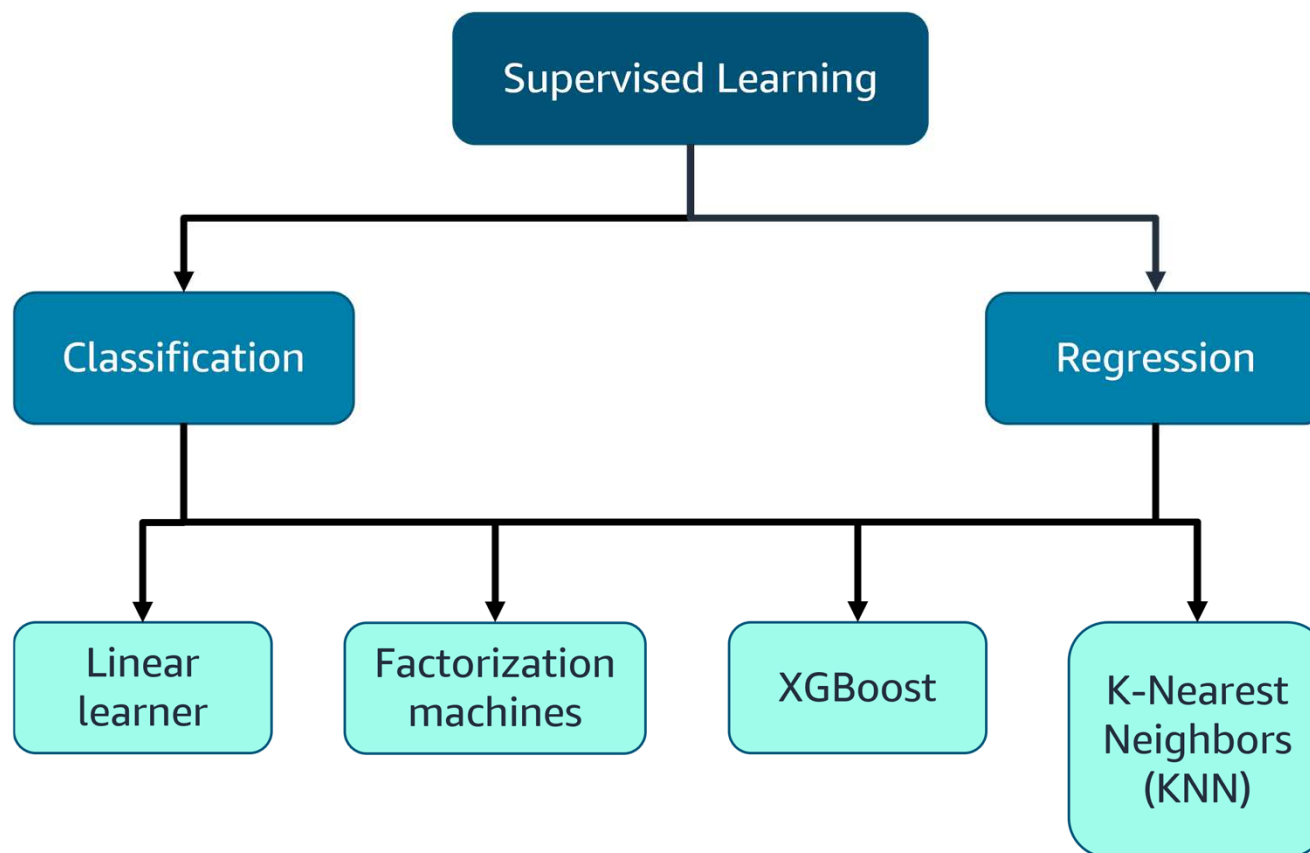


Sources of ML Models

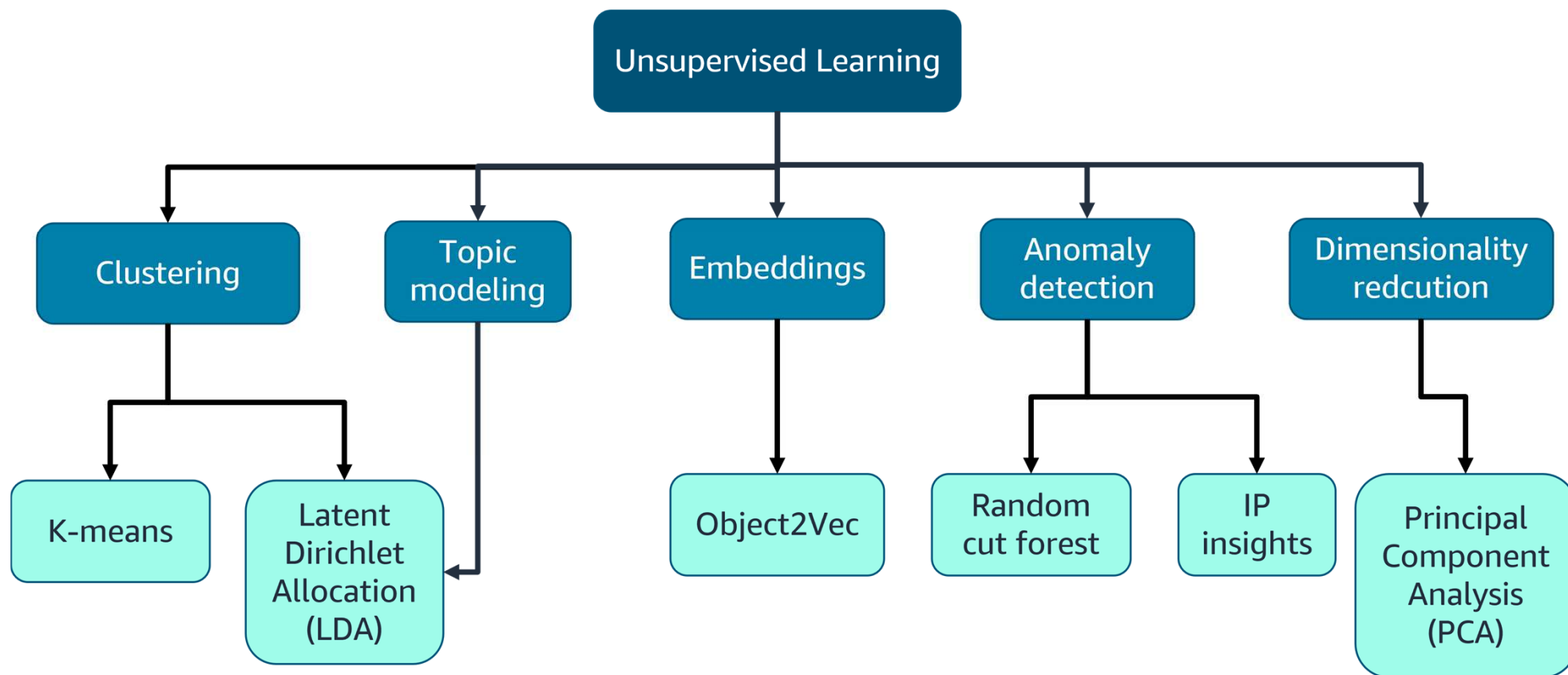
Ways to use SageMaker to build your ML model:

- Pre-trained models require the least effort and are models ready to deploy or to fine-tune and deploy using **SageMaker JumpStart**.
- Built-in models available in SageMaker require more effort and scale if the dataset is large and significant resources are needed to train and deploy the model.
- If there is no built-in solution that works, try to develop one that uses pre-made images for machine learning and deep learning frameworks for supported frameworks such as scikit-learn, TensorFlow, PyTorch, MXNet, or Chainer.
- You can build your own custom Docker image that is configured to install the necessary packages or software.

SageMaker Built-in Algorithms

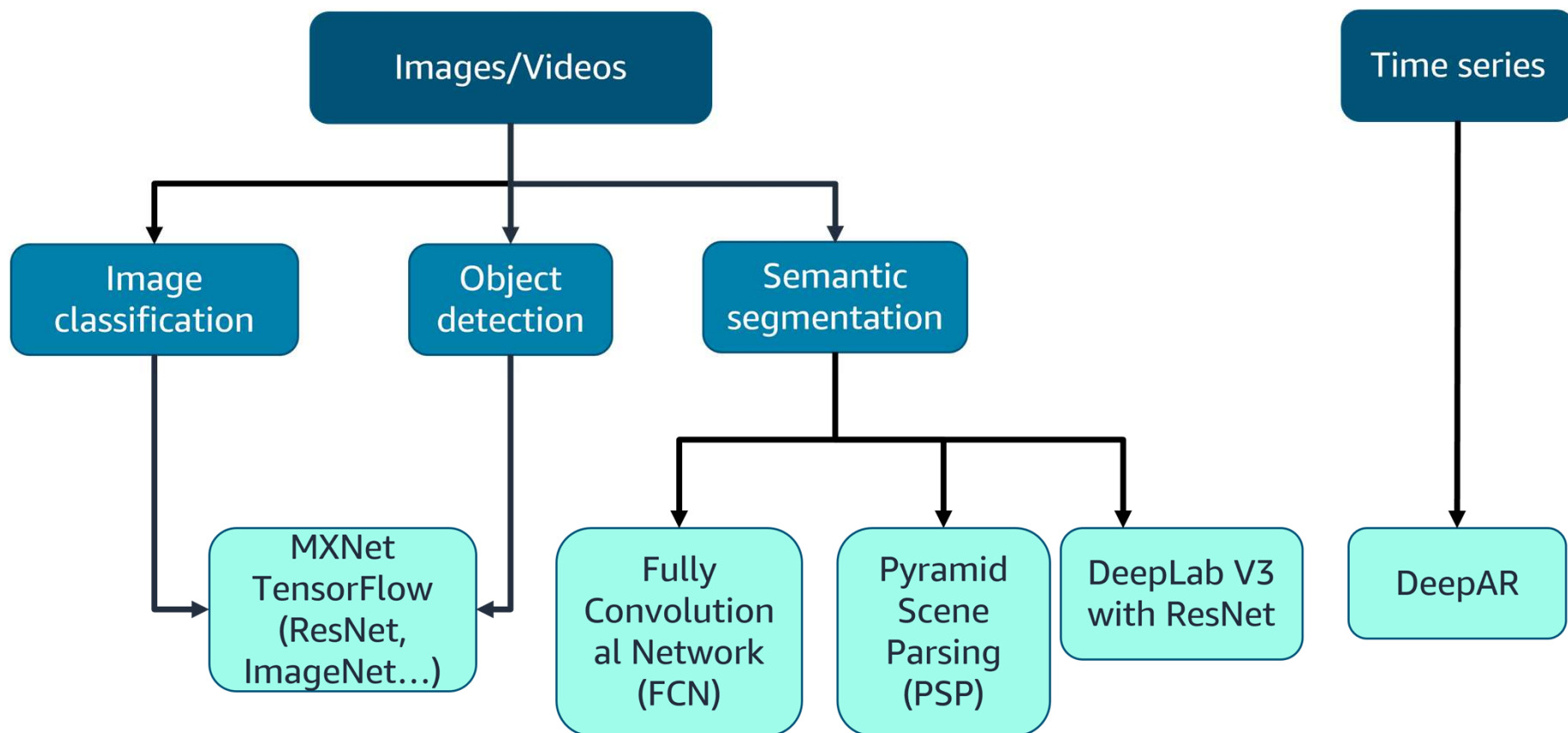


SageMaker Built-in Algorithms

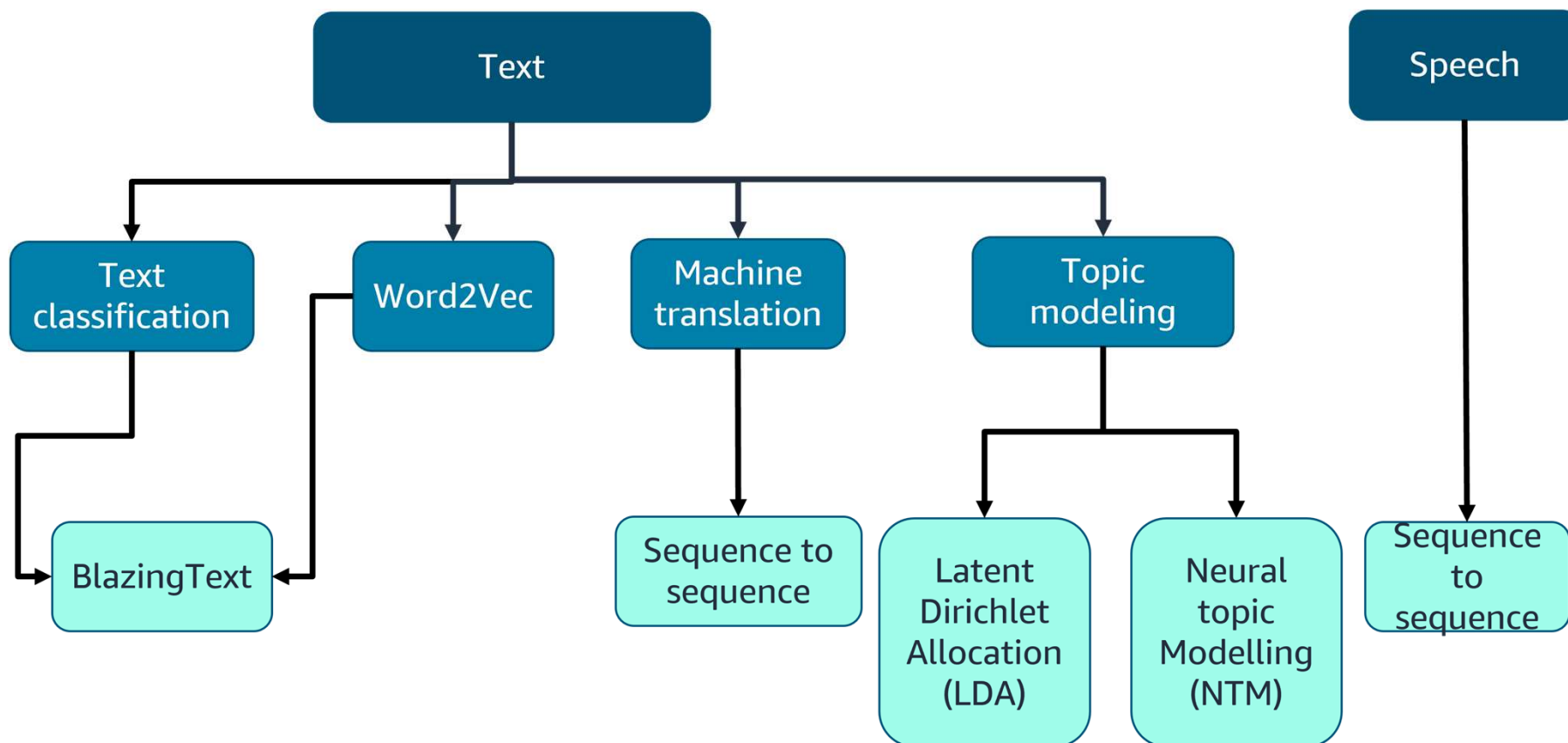


SageMaker Built-in Algorithms

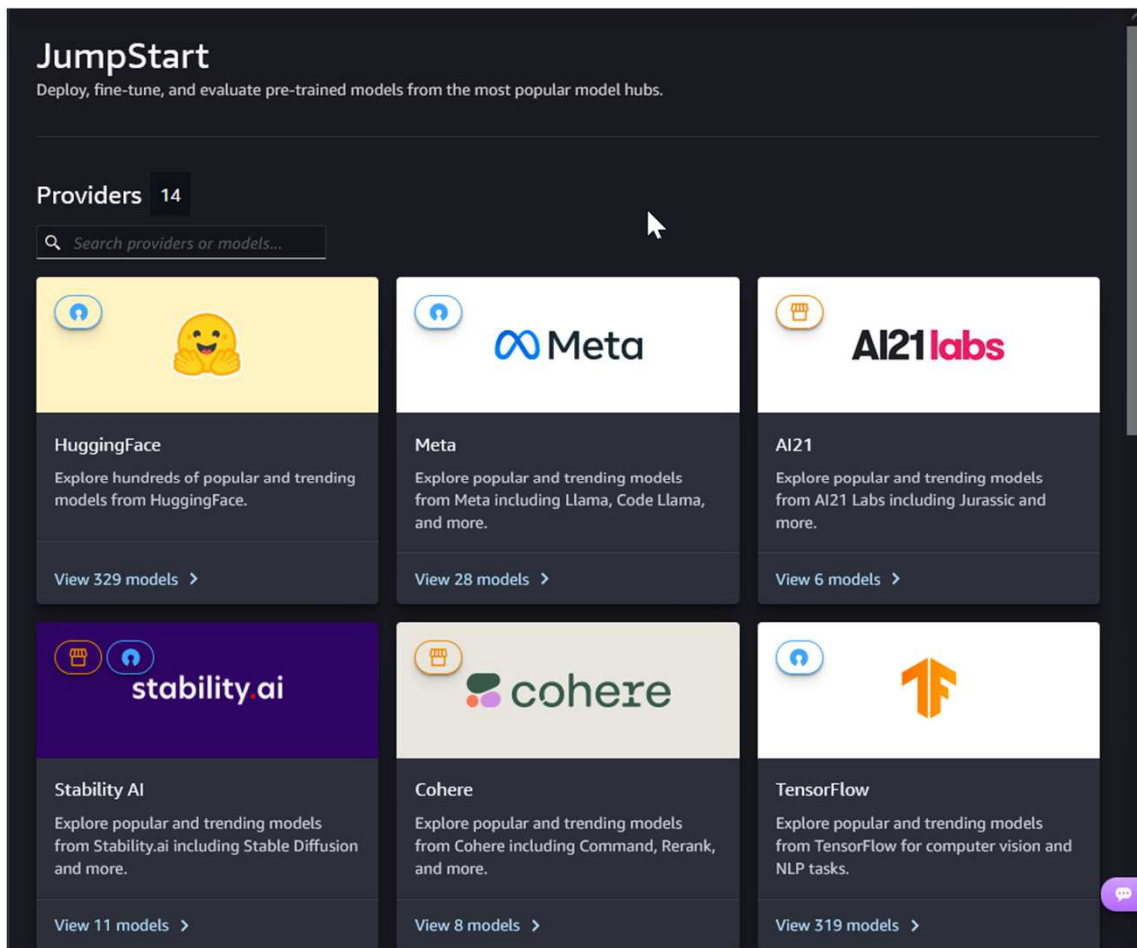
Image Processing



SageMaker Built-in Algorithms



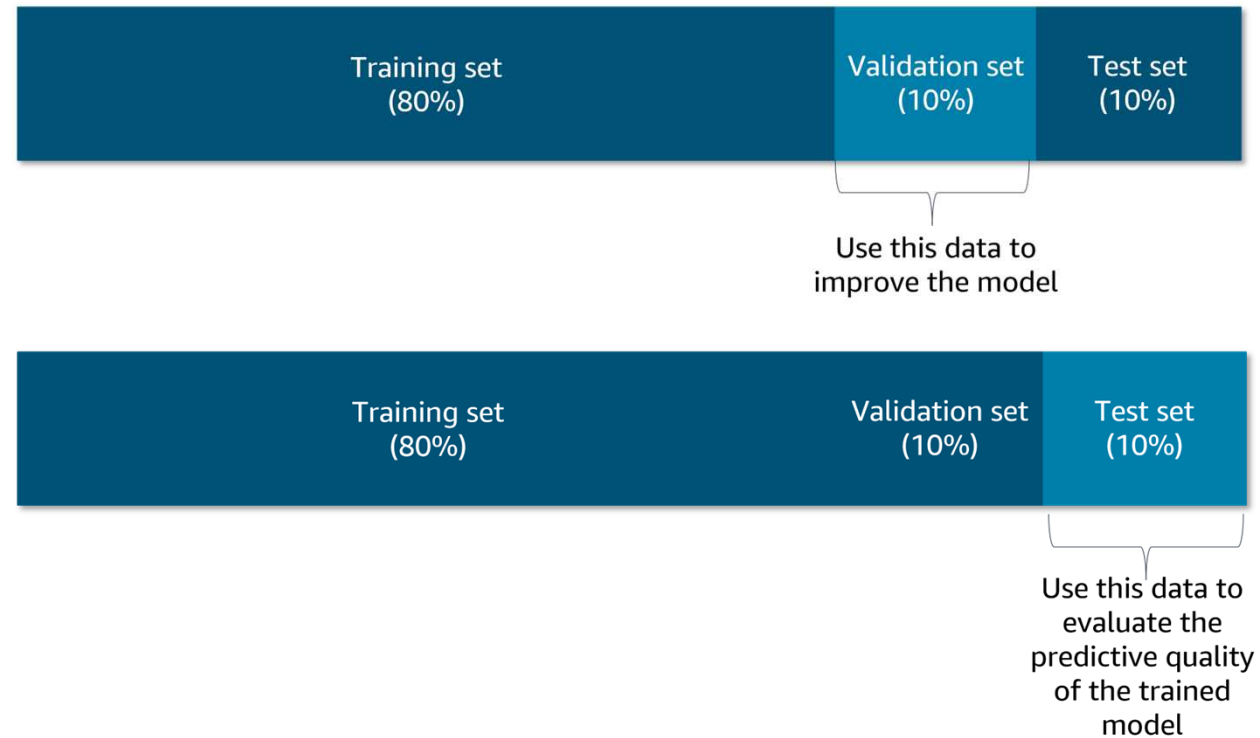
SageMaker Jumpstart



- SageMaker JumpStart provides pretrained open source models from leading providers for a range of problem types to help you get started with machine learning.
- Provides solution templates that set up infrastructure for common use cases and runnable example notebooks for machine learning with SageMaker.

Machine Learning Models Performance Evaluation

- Evaluation occurs after a model is trained.
- The data you use is partitioned into three parts: training set, validation set, and test set.



Model Fit

- Overfitting

Model performs well on the training data but does not perform well on the evaluation data. This is because the model memorized the data it has seen and is unable to generalize to unseen examples.

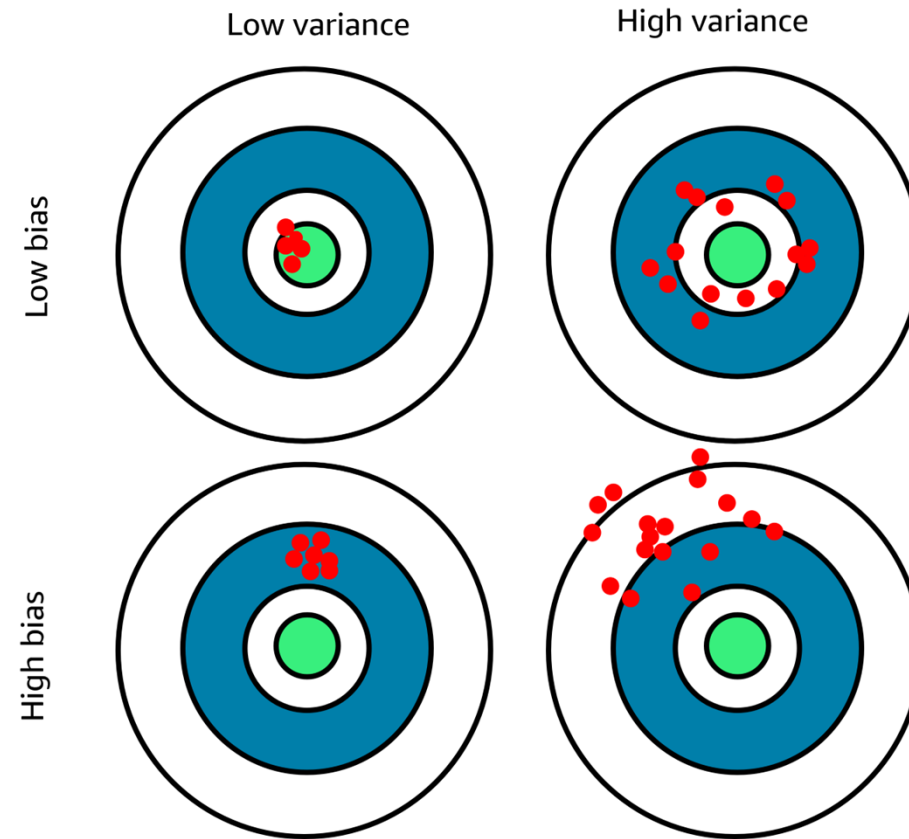
- Underfitting

Performs poorly on the training data. Model is unable to capture the relationship between the input examples (often called X) and the target values (often called Y).

- Balanced

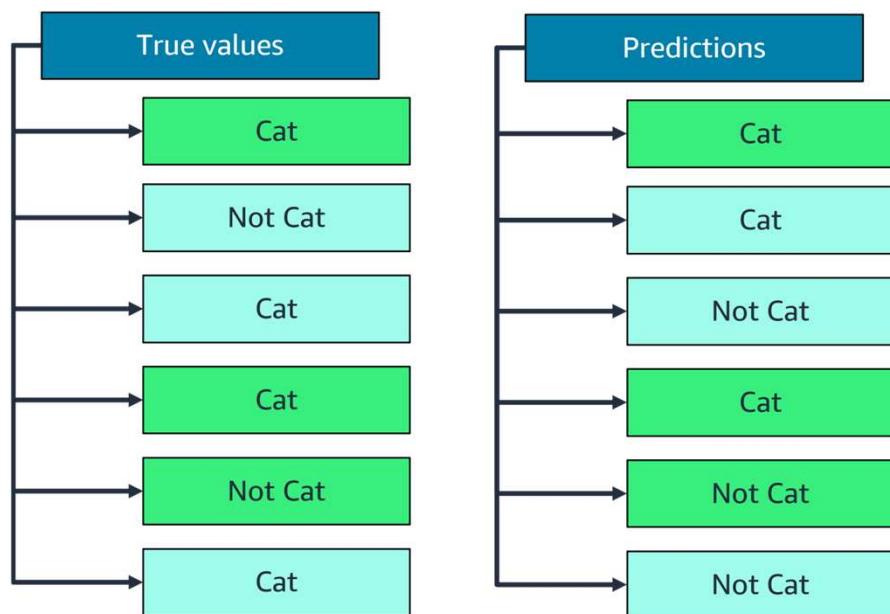
The model is balanced when it is not overfit or underfit to the training data.

Bias and variance



Classification Problems

Classification Metrics	Regression Metrics
Accuracy	Mean squared error
Precision	R Squared
Recall	
F1 score	
AUC-ROC	



Predicted class		P	N
Actual class	P	True Positive (TP)	False Negative (FN)
	N	False Positive (FP)	True Negative (TN)

Regression Problem Metrics

Mean squared error

- The general purpose of mean squared error (MSE) is the same as the classification metrics. You determine the prediction from the model and compare the difference between the prediction and the actual outcome.

R squared

- R squared is another commonly used metric with linear regression problems. R squared explains the fraction of variance accounted for by the model. It's like a percentage, reporting a number from 0 to 1. When R squared is close to 1, it usually indicates that a lot of the variance in the data can be explained by the model itself.

Model Deployment Types

- Self-Hosted API

Advantages of self-hosted APIs include greater control over the infrastructure, potential cost savings (depending on usage), and the ability to customize the deployment environment. However, this approach requires more operational overhead and responsibility for managing and maintaining the infrastructure.

- Managed

Cloud-based services that provide a fully managed environment for deploying and hosting your ML models as APIs. SageMaker is an example. These services abstract away the underlying infrastructure management so you can focus on building and deploying your models. API

SageMaker

SageMaker to deploy a model to get predictions in several ways.

- **Real-time**

Real-time inference is ideal for inference workloads where you have real-time, interactive and low latency requirements.

- **Batch transform**

Used for large datasets inference and don't need a persistent endpoint. Used also to preprocess datasets to remove noise or bias that interferes with training or inference from your dataset.

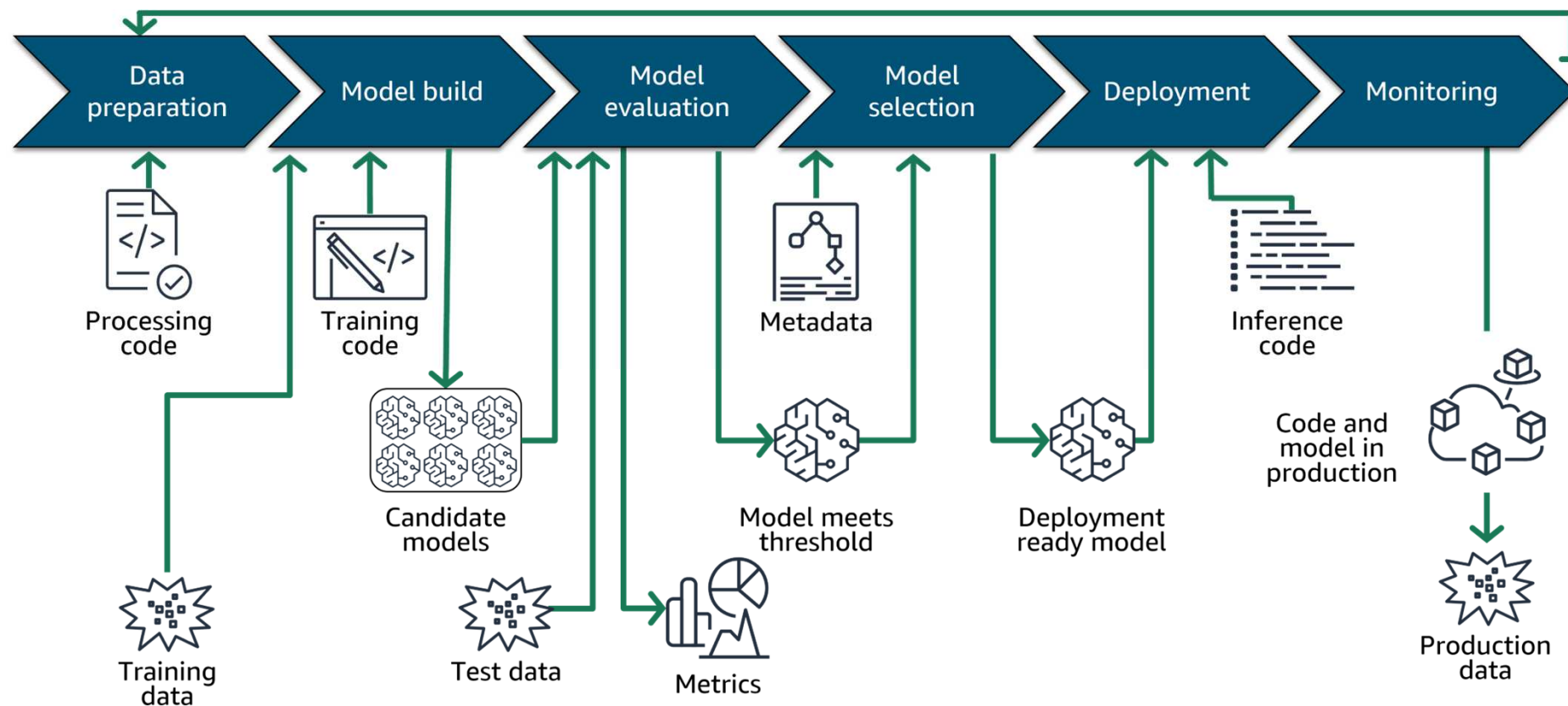
- **Asynchronous**

Queues incoming requests and processes them asynchronously. Ideal for requests with large payload sizes (up to 1GB), long processing times (up to one hour) and near real-time latency requirements

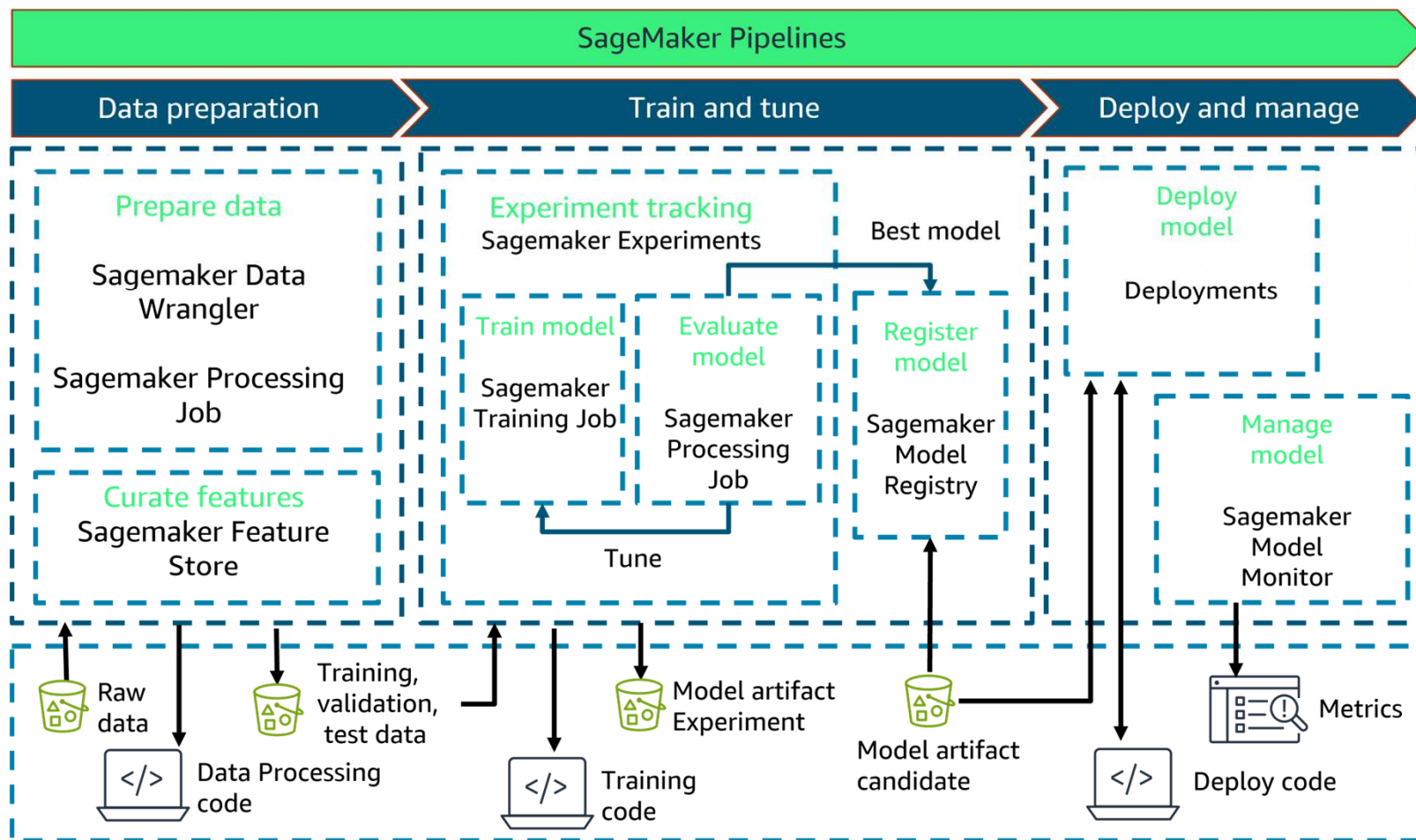
- **Serverless**

Ideal for workloads that have idle periods between traffic spurts and can tolerate cold starts without configuring or managing any of the underlying infrastructure.

ML Lifecycle and MLOps



AWS Services for MLOps



Q1. What is the goal of feature engineering in the machine learning lifecycle?

- A. To collect and clean data
- B. To transform data and create variables for the model
- C. To calculate the model performance
- D. To maintain the desired level of performance

Q1. What is the goal of feature engineering in the machine learning lifecycle?

- A. To collect and clean data
- B. To transform data and create variables for the model**
- C. To calculate the model performance
- D. To maintain the desired level of performance

Q2. Knowledge Check

A small e-commerce company wants to use machine learning to improve their churn prediction. However, the company does not have a dedicated data science team and is looking for a low-code or no-code solution to get started with machine learning. Which Amazon SageMaker feature would be the most suitable for the company's requirements?

- A. Amazon SageMaker Canvas
- B. Amazon SageMaker JumpStart
- C. Amazon SageMaker Data Wrangler
- D. Amazon SageMaker Feature Store

Q2. Knowledge Check

A small e-commerce company wants to use machine learning to improve their churn prediction. However, the company does not have a dedicated data science team and is looking for a low-code or no-code solution to get started with machine learning. Which Amazon SageMaker feature would be the most suitable for the company's requirements?

- A. Amazon SageMaker Canvas**
- B. Amazon SageMaker JumpStart
- C. Amazon SageMaker Data Wrangler
- D. Amazon SageMaker Feature Store

Q3.

A company wants to implement a proof of concept about price predictions for a set of items. They are starting from scratch and want a solution with minimal operational overhead.

Which implementation should the company use?

- A. Amazon SageMaker JumpStart
- B. A custom docker image
- C. A custom script using a supported third-party framework
- D. A custom script using a built-in SageMaker algorithm

Q3.

A company wants to implement a proof of concept about price predictions for a set of items. They are starting from scratch and want a solution with minimal operational overhead.

Which implementation should the company use?

- A. Amazon SageMaker JumpStart**
- B. A custom docker image
- C. A custom script using a supported third-party framework
- D. A custom script using a built-in SageMaker algorithm

Q4.

A company develops a machine learning model to predict the price of a house based on its features, such as the number of bedrooms, bathrooms, square footage, and location. The model is trained on a dataset of historical house sales, and its performance is evaluated using various metrics. However, the model is found to perform well on the training data but poorly on new, unseen data.

Which option describes the fit of model?

- A. It overfits.
- B. It underfits.
- C. It has a low variance.
- D. It has a high bias.

Q4.

A company develops a machine learning model to predict the price of a house based on its features, such as the number of bedrooms, bathrooms, square footage, and location. The model is trained on a dataset of historical house sales, and its performance is evaluated using various metrics. However, the model is found to perform well on the training data but poorly on new, unseen data.

Which option describes the fit of model?

- A. It overfits.**
- B. It underfits.
- C. It has a low variance.
- D. It has a high bias.

Q5.

A real-time object detection model has been developed by a team of developers for a self-driving car company to identify various objects in the environment for their autonomous vehicle. The model must be deployed in a way that ensures minimal latency and must be capable of interacting with the vehicle's onboard systems. Which AWS deployment options should the team choose to deploy the model to achieve minimal latency while enabling real-time interactions with the onboard systems?

- A. Real-time
- B. Asynchronous
- C. Batch
- D. Serverless

Q5.

A real-time object detection model has been developed by a team of developers for a self-driving car company to identify various objects in the environment for their autonomous vehicle. The model must be deployed in a way that ensures minimal latency and must be capable of interacting with the vehicle's onboard systems. Which AWS deployment options should the team choose to deploy the model to achieve minimal latency while enabling real-time interactions with the onboard systems?

- A. **Real-time**
- B. Asynchronous
- C. Batch
- D. Serverless

Questions?